

Acceso a ficheros en Java

Persistencia de datos, lectura y escritura de archivos, objetos y formatos de intercambio

Índice de contenidos

Desde los primeros archivos hasta el almacenamiento de objetos y el intercambio de información entre aplicaciones.

1. Qué significa guardar datos

2. Archivos, carpetas y rutas

3. Leer y escribir ficheros de texto

4. Altas, consultas, modificaciones y bajas

5. Copias de seguridad y recuperación

6. Almacenar objetos en ficheros binarios

7. Acceso aleatorio a registros de tamaño fijo

8. Intercambio de datos con XML

9. Intercambio de datos con JSON

10. Organización del acceso a datos

11. Proyecto de unidad: gestor de clubes

1. Qué significa guardar datos

Hasta ahora, al programar en Java, has utilizado **variables y objetos** que viven mientras el programa está ejecutándose. Si creas una lista de clubes, cierras el programa y lo vuelves a abrir, esa lista desaparece salvo que hayas guardado sus datos en algún lugar.

Llamamos **persistencia** a la capacidad de conservar datos después de que termine una ejecución. En esta unidad aprenderás a conseguirla mediante **ficheros**, es decir, archivos almacenados en el disco del ordenador. Más adelante, en otras unidades, utilizarás una base de datos.

Imagina una aplicación que gestiona clubes deportivos. El usuario añade un club, consulta su ciudad y cambia su nombre. Para que esos cambios sigan disponibles al día siguiente, el programa debe escribirlos en un fichero y leerlos cuando vuelva a iniciarse.

Esquema de trabajo

DATOS

```
Datos introducidos → objetos y variables Java
                    ↓ guardar
                    fichero en el disco
                    ↓ recuperar
                    objetos y variables Java
```

1.1 Dos formas habituales de guardar información

Un **fichero de texto** almacena caracteres legibles, como `1;Granada CF;Granada`. Lo puedes abrir con un editor de texto para comprobar qué se ha guardado. Un **fichero binario** contiene bytes con una estructura definida por el programa; al abrirlo con un editor normal no suele verse como una tabla de palabras. Trabajaremos con ambos.

También aprenderás a diferenciar **acceso secuencial** (leer los registros uno detrás de otro) y **acceso aleatorio** (saltar directamente a una posición conocida dentro del fichero).

2. Archivos, carpetas y rutas

Un **archivo** o *fichero* es una unidad de información guardada en un dispositivo. Una **carpeta**, también llamada *directorio*, organiza archivos y otras carpetas. Una **ruta** indica dónde se encuentra un archivo o una carpeta.

Por ejemplo, la ruta `datos/clubes.txt` significa que existe un archivo llamado `clubes.txt` dentro de la carpeta `datos`.

DATOS

```
AD_UD1_Clubes/
├── src/
│   └── Main.java
└── datos/
    └── clubes.txt
```

La ruta `datos/clubes.txt` es **relativa**: se interpreta desde el directorio de trabajo con el que se ejecuta la aplicación. En cambio, una ruta **absoluta** indica la ubicación completa del archivo en el sistema operativo, por ejemplo `C:\\Usuarios\\Alumno\\AD_UD1_Clubes\\datos\\clubes.txt` en un ordenador Windows. Una ruta absoluta depende de ese equipo y no conviene escribirla directamente en el código de clase.

2.1 Cómo representa Java una ruta: `Path`

La clase `Path`, del paquete `java.nio.file`, representa una ruta. **Representar una ruta no significa que el fichero exista todavía**. La clase `Files`, del mismo paquete, permite hacer operaciones reales sobre el disco: comprobar si existe una ruta, crear directorios, leer y escribir.

Antes de utilizar estos métodos, observa qué hace cada uno:

Elemento	Significado
<code>Path.of("datos")</code>	Construye un objeto <code>Path</code> que representa la carpeta <code>datos</code> .
<code>carpeta.resolve("clubes.txt")</code>	Construye otra ruta añadiendo <code>clubes.txt</code> a <code>carpeta</code> .
<code>archivo.toAbsolutePath()</code>	Obtiene la ruta absoluta correspondiente a <code>archivo</code> . No crea el fichero ni comprueba que exista.
<code>Files.exists(archivo)</code>	Devuelve <code>true</code> si la ruta existe en el sistema de ficheros y <code>false</code> si no se ha podido confirmar su existencia.
<code>Files.notExists(archivo)</code>	Comprueba que la ruta no existe. No es exactamente lo mismo que negar <code>exists</code> si faltan permisos.
<code>Files.createDirectories(carpeta)</code>	Crea la carpeta y sus carpetas superiores si hacen falta. Si ya existe como directorio, no la borra.
<code>Files.createFile(archivo)</code>	Crea un archivo vacío; falla si ya existe.
<code>Files.size(archivo)</code>	Devuelve su tamaño en bytes .

En Java, un **método** es una operación que puede ejecutarse sobre un objeto o sobre una clase. En `archivo.toAbsolutePath()`, el método pertenece al objeto `archivo`; en `Files.exists(archivo)`, se llama sobre la clase `Files` y se le pasa el objeto `archivo` como argumento. El resultado de una condición como `Files.exists(...)` es un valor **booleano** (`true` o `false`).

2.2 Qué ocurre si Java no puede acceder al disco

Las operaciones con archivos pueden fallar: la carpeta no existe, faltan permisos o el disco está lleno. Java puede comunicar muchos de estos problemas mediante una excepción llamada `IOException`. Un bloque `try` contiene el código que podría fallar; `catch` indica qué debe hacer el programa si aparece el error. El método `getMessage()` del error proporciona información para mostrarla al usuario.

Ejemplo: crear una carpeta y un fichero

Crea un proyecto Java llamado `AD_UD1_Clubes` y añade una clase `Main`. Ejecuta el siguiente código desde el directorio del proyecto:

JAVA

```
import java.io.IOException;
import java.nio.file.Files;
import java.nio.file.Path;

public class Main {
    public static void main(String[] args) {
        Path carpeta = Path.of("datos");
        Path archivo = carpeta.resolve("clubes.txt");

        try {
            Files.createDirectories(carpeta);

            if (Files.notExists(archivo)) {
                Files.createFile(archivo);
            }

            System.out.println("Ruta: " + archivo.toAbsolutePath());
            System.out.println("¿Existe?: " + Files.exists(archivo));
            System.out.println("Tamaño: " + Files.size(archivo) + " bytes");
        } catch (IOException e) {
            System.err.println("No se pudo acceder al archivo: "
                + e.getMessage());
        }
    }
}
```

Lee el código de arriba abajo: primero se construyen dos rutas; después se crea la carpeta; solo si el archivo no existe se crea vacío; por último se muestran su ubicación, existencia y tamaño. Si lo ejecutas una segunda vez, el fichero creado en la primera ejecución **no se borra ni se sustituye**.

Actividades

1. Ejecuta el programa dos veces y localiza el archivo real en el explorador de archivos.
2. Abre `clubes.txt`, escribe una línea de texto, guarda el fichero y vuelve a ejecutar Java. Explica por qué el tamaño ya no es cero.
3. Crea desde Java la subcarpeta `datos/copias` y, dentro de ella, un archivo vacío `respaldo.txt`. Comprueba que volver a ejecutar no sobrescribe su contenido.

2.3 Consultar el contenido de un directorio

A veces necesitamos conocer los ficheros que hay dentro de una carpeta. `Files.newDirectoryStream(carpeta)` abre un recorrido de sus elementos. El tipo `DirectoryStream<Path>` representa ese recorrido y el bucle `for (Path elemento : elementos)` visita cada entrada.

Para distinguir archivos de carpetas, `Files.isRegularFile(elemento)` comprueba si una ruta corresponde a un archivo ordinario. `elemento.getFileName()` devuelve solo el nombre final, por ejemplo `clubes.txt`, en lugar de la ruta completa. La instrucción `try (...)` de este ejemplo cierra automáticamente el recurso cuando acaba el bloque; esta forma de escribir `try` se llama **try-with-resources**.

JAVA

```
import java.io.IOException;
import java.nio.file.DirectoryStream;
import java.nio.file.Files;
import java.nio.file.Path;

public class ListarArchivos {
    public static void main(String[] args) {
        Path carpeta = Path.of("datos");

        try (DirectoryStream<Path> elementos =
            Files.newDirectoryStream(carpeta)) {
            for (Path elemento : elementos) {
                if (Files.isRegularFile(elemento)) {
                    System.out.println(elemento.getFileName());
                }
            }
        } catch (IOException e) {
            System.err.println(e.getMessage());
        }
    }
}
```

Si el directorio no existe, el programa lo comunicará a través del `catch`.

3. Leer y escribir ficheros de texto

Un fichero de texto permite guardar información como caracteres. Antes de escribirlo debemos acordar **cómo se representará cada dato**, porque Java no puede adivinar por sí solo dónde termina el nombre de un club y empieza su ciudad.

Usaremos inicialmente **un registro por línea** y un punto y coma (`;`) para separar los campos:

DATOS

```
1;Granada CF;Granada
2;Málaga CF;Málaga
3;Cádiz CF;Cádiz
```

En este ejemplo, un **registro** es una línea con los datos de un club, y cada dato separado por `;` es un **campo**. Mientras utilizemos este formato sencillo, no escribiremos `;` dentro de los nombres o ciudades. Más adelante verás formatos que resuelven mejor estas limitaciones.

3.1 Qué significa codificación UTF-8

El texto se guarda en el disco como bytes. Una **codificación** define cómo representar cada carácter mediante esos bytes. Utilizaremos `UTF-8`, que admite tildes, ñes y otros caracteres. En Java, `StandardCharsets.UTF_8` permite indicar explícitamente esa codificación para evitar diferencias entre equipos.

3.2 Qué hace cada clase antes de escribir

Elemento	Para qué sirve
<code>BufferedWriter</code>	Escribe texto de forma cómoda, normalmente acumulándolo antes de enviarlo al disco.
<code>Files.newBufferedWriter(ruta, UTF_8)</code>	Abre el archivo para escribir en UTF-8. Por defecto, crea o sustituye el contenido existente.
<code>salida.write(texto)</code>	Escribe los caracteres indicados.
<code>salida.newLine()</code>	Escribe un salto de línea compatible con el sistema.
<code>try (BufferedWriter salida = ...)</code>	Cierra el escritor automáticamente, también si ocurre una excepción.

Importante: abrir un archivo para escribir no significa necesariamente *añadir*. Si quieres conservar sus líneas anteriores, debes abrirlo en el modo adecuado, que veremos más adelante.

Antes de ejecutar el ejemplo, fíjate en `ruta.getParent()`: devuelve la ruta de la carpeta que contiene el archivo, en este caso `datos`. Así podremos crearla aunque no exista todavía.

Ejemplo: guardar tres clubes

JAVA

```
import java.io.BufferedWriter;
import java.io.IOException;
import java.nio.charset.StandardCharsets;
import java.nio.file.Files;
import java.nio.file.Path;

public class GuardarClubes {
    public static void main(String[] args) {
        Path ruta = Path.of("datos", "clubes.txt");

        try {
            Files.createDirectories(ruta.getParent());

            try (BufferedWriter salida = Files.newBufferedWriter(
                ruta, StandardCharsets.UTF_8)) {
                salida.write("1;Granada CF;Granada");
                salida.newLine();
                salida.write("2;Málaga CF;Málaga");
                salida.newLine();
                salida.write("3;Cádiz CF;Cádiz");
                salida.newLine();
            }
        } catch (IOException e) {
            System.err.println("No se pudo guardar: " + e.getMessage());
        }
    }
}
```

Al terminar, abre `datos/clubes.txt` con un editor de texto y comprueba que cada club ocupa una línea distinta.

3.3 Qué ocurre al leer un fichero

Para recuperar las líneas necesitamos un `BufferedReader`. `Files.newBufferedReader(ruta, UTF_8)` abre un lector de texto; `entrada.readLine()` devuelve la siguiente línea **sin su salto final**, y devuelve `null` cuando ya no quedan líneas. Ese `null` es la condición que detiene el bucle `while`.

Antes de leer conviene comprobar que el fichero existe. Aquí utilizamos `Files.notExists(ruta)` para mostrar un mensaje comprensible en vez de dejar que el intento de apertura falle sin explicación.

JAVA

```
import java.io.BufferedReader;
import java.io.IOException;
import java.nio.charset.StandardCharsets;
import java.nio.file.Files;
import java.nio.file.Path;

public class LeerClubes {
    public static void main(String[] args) {
        Path ruta = Path.of("datos", "clubes.txt");

        if (Files.notExists(ruta)) {
            System.out.println("Todavía no hay un archivo de clubes.");
            return;
        }

        try (BufferedReader entrada = Files.newBufferedReader(
            ruta, StandardCharsets.UTF_8)) {
            String linea;
            while ((linea = entrada.readLine()) != null) {
                System.out.println(linea);
            }
        } catch (IOException e) {
            System.err.println("No se pudo leer: " + e.getMessage());
        }
    }
}
```

Comprueba lo aprendido: guarda un cuarto club a mano, ejecuta `LeerClubes` y verifica que muestra las cuatro líneas. Después renombra el archivo para simular que no existe y observa el mensaje que produce el programa.

3.4 Añadir una línea sin borrar las anteriores

Hasta aquí, `newBufferedWriter` sustituía el contenido de un archivo existente. Para abrirlo en modo **añadir al final** utilizamos opciones de apertura: `StandardOpenOption.CREATE` significa «crea el archivo si falta» y `StandardOpenOption.APPEND` significa «escribe a continuación de los datos existentes».

JAVA

```
import java.io.BufferedWriter;
import java.io.IOException;
import java.nio.charset.StandardCharsets;
import java.nio.file.Files;
import java.nio.file.Path;
import java.nio.file.StandardOpenOption;

public class AnadirClub {
    public static void main(String[] args) {
        Path ruta = Path.of("datos", "clubes.txt");
        try {
            Files.createDirectories(ruta.getParent());
            try (BufferedWriter salida = Files.newBufferedWriter(
                ruta, StandardCharsets.UTF_8,
                StandardOpenOption.CREATE,
                StandardOpenOption.APPEND)) {
                salida.write("4;Almería;Almería");
                salida.newLine();
            }
        } catch (IOException e) {
            System.err.println(e.getMessage());
        }
    }
}
```

Si ejecutas `AnadirClub` tres veces, **añadirá la misma línea tres veces**. Java no sabe que ya existe ese identificador. Evitar registros duplicados será responsabilidad de nuestro programa.

Actividades

1. Pide por teclado el identificador, el nombre y la ciudad de un club; guarda la información respetando el formato `id;nombre;ciudad`.
2. Lee el fichero y muestra cuántos registros hay. No cuentes las líneas vacías.
3. Comprueba qué ocurre cuando intentas leer un fichero que todavía no existe y cuando vuelves a ejecutar el programa que añade registros.

4. Altas, consultas, modificaciones y bajas

Las cuatro operaciones habituales sobre un conjunto de datos se resumen con el acrónimo **CRUD**, del inglés *Create, Read, Update, Delete*: crear, leer, modificar y borrar. En nuestros ficheros equivalen a dar de alta un club, buscarlo, modificar su ciudad o eliminarlo.

En un fichero de texto con líneas de longitud variable no podemos cambiar sin más un nombre de cinco letras por otro de veinte ocupando los mismos bytes: podríamos pisar el siguiente registro. Para **modificar o borrar** trabajaremos con esta secuencia: leer todo el archivo, cambiar la información en memoria y volver a escribir el fichero completo.

4.1 Convertir una línea en campos

`linea.split(";", -1)` separa una cadena por el delimitador `;` y crea un array de `String`. El segundo argumento, `-1`, hace que Java conserve los campos vacíos al final. `Integer.parseInt(texto)` intenta convertir texto en un número entero y puede lanzar `NumberFormatException` si el contenido no es válido. Antes de usar los campos, comprobaremos que hay exactamente tres.

JAVA

```
String linea = "2;Málaga CF;Málaga";
String[] campos = linea.split(";", -1);

if (campos.length == 3) {
    try {
        int id = Integer.parseInt(campos[0]);
        String nombre = campos[1];
        String ciudad = campos[2];
        System.out.println(id + " -> " + nombre + " (" + ciudad + ")");
    } catch (NumberFormatException e) {
        System.out.println("El ID no contiene un número válido.");
    }
}
```

`campos[0]`, `campos[1]` y `campos[2]` son, respectivamente, identificador, nombre y ciudad. Si el archivo incluye una línea sin tres campos, **no debemos usar** `campos[2]` porque no existirá.

4.2 Leer todas las líneas en memoria

La clase `List<String>` representa una secuencia de cadenas, y `ArrayList<String>` permite construir una lista que puede crecer y modificarse. Para ficheros pequeños, `Files.readAllLines(ruta, UTF_8)` devuelve todas sus líneas como una lista. En un fichero enorme usaríamos lectura progresiva para no cargarlo entero; aquí los datos son reducidos.

`Files.write(ruta, lineas, UTF_8)` escribe las líneas de una lista: crea el fichero si falta y, con esta llamada, **reemplaza** el contenido anterior. Por eso una modificación exige conservar en la lista todas las líneas que no queremos cambiar.

Ejemplo: modificar la ciudad del club con ID 2

JAVA

```
import java.io.IOException;
import java.nio.charset.StandardCharsets;
import java.nio.file.Files;
import java.nio.file.Path;
import java.util.ArrayList;
import java.util.List;

public class ModificarCiudad {
    public static void main(String[] args) {
        Path ruta = Path.of("datos", "clubes.txt");

        try {
            if (Files.notExists(ruta)) {
                System.out.println("No existe el archivo de clubes.");
                return;
            }

            List<String> originales = Files.readAllLines(
                ruta, StandardCharsets.UTF_8);
            List<String> nuevas = new ArrayList<>();
```

```

        boolean encontrado = false;

        for (String linea : originales) {
            String[] campos = linea.split(";", -1);
            if (campos.length == 3 && campos[0].equals("2")) {
                nuevas.add(campos[0] + ";" + campos[1] + ";Nueva ciudad");
                encontrado = true;
            } else {
                nuevas.add(linea);
            }
        }

        if (encontrado) {
            Files.write(ruta, nuevas, StandardCharsets.UTF_8);
            System.out.println("Ciudad actualizada.");
        } else {
            System.out.println("No existe el club con ID 2.");
        }
    } catch (IOException e) {
        System.err.println("No se pudo modificar: " + e.getMessage());
    }
}
}

```

Aquí `equals("2")` compara el **contenido** de la cadena con el texto "2". `nuevas.add(...)` agrega una línea a la lista y la variable booleana `encontrado` permite saber si hemos modificado algún registro. Solo escribimos el archivo si se encontró el club: buscar un ID inexistente no tiene por qué reescribir los datos.

4.3 Evitar datos duplicados y validar

Antes de dar de alta un club, recorre el archivo y comprueba que no existe otro con el mismo ID. Para comparar nombres sin que influyan las mayúsculas y los espacios extremos, `trim()` elimina esos espacios y `equalsIgnoreCase(...)` compara el texto sin distinguir entre mayúsculas y minúsculas.

Cuando el programa **importa información**, también debe validar el número de campos, el tipo de ID y los campos obligatorios. Un registro defectuoso no debe provocar la pérdida de los registros válidos.

Actividades

1. Implementa un menú con cuatro opciones: añadir, buscar por ID, modificar ciudad y borrar por ID.
2. Impide que dos clubes tengan el mismo ID. Después impide también que se repita el nombre aunque cambien las mayúsculas.
3. Para borrar, crea una lista nueva con todas las líneas **menos** la correspondiente al ID solicitado y guarda solo si el ID existía.
4. Prueba los siguientes casos: archivo inexistente, archivo vacío, club duplicado, ID no numérico e ID que no aparece en el fichero.

5. Copias de seguridad y recuperación

Una **copia de seguridad** es una segunda versión de los datos que permite recuperarlos si ocurre un error. Antes de una modificación que reescribe un fichero, conviene guardar una copia de la versión anterior.

`Files.copy(origen, destino)` copia un fichero; la opción `StandardCopyOption.REPLACE_EXISTING` permite reemplazar un archivo de destino que ya exista. `Files.move(origen, destino, ...)` cambia de ubicación un archivo: no equivale a crear una copia. `Files.deleteIfExists(ruta)` elimina un archivo si existe; como el borrado puede ser irreversible, no lo utilizaremos sin una confirmación previa del usuario.

Ejemplo: guardar un respaldo

JAVA

```
import java.io.IOException;
import java.nio.file.Files;
import java.nio.file.Path;
import java.nio.file.StandardCopyOption;

public class CrearRespaldo {
    public static void main(String[] args) {
        Path origen = Path.of("datos", "clubes.txt");
        Path destino = Path.of("datos", "copias", "clubes.bak");

        try {
            if (Files.notExists(origen)) {
                System.out.println("No hay datos que copiar.");
                return;
            }
            Files.createDirectories(destino.getParent());
            Files.copy(origen, destino,
                StandardCopyOption.REPLACE_EXISTING);
            System.out.println("Copia creada: " + destino.toAbsolutePath());
        } catch (IOException e) {
            System.err.println("No se pudo crear la copia: " + e.getMessage());
        }
    }
}
```

Este ejemplo mantiene **una sola copia**, porque una nueva ejecución sustituye `clubes.bak`. Si quieres conservar varias, puedes añadir al nombre un identificador distinto, por ejemplo una fecha y hora. La recuperación consiste en copiar el respaldo hacia el archivo de datos, pero **antes debes comprobar que la copia es válida y solicitar confirmación**, porque esa operación puede sustituir información más reciente.

Actividades

1. Integra una opción «Crear copia de seguridad» en el gestor de clubes.
2. Añade «Restaurar copia»: si no existe el respaldo, muestra un aviso; si existe, pide confirmación y conserva una copia de la versión que se va a sustituir.
3. Introduce una línea errónea en una copia de prueba, detecta el problema y evita reemplazar el archivo original por datos que no puedan interpretarse.

6. Almacenar objetos en ficheros binarios

Hasta ahora hemos guardado cadenas y hemos tenido que separar sus campos al recuperarlas. También es posible almacenar un objeto Java en un fichero binario. El proceso de **serialización** convierte el objeto en bytes; la **deserialización** lo reconstruye cuando se vuelve a leer el fichero.

La interfaz `Serializable` indica que una clase admite este mecanismo. `ObjectOutputStream` escribe objetos en un flujo binario y `ObjectInputStream` los recupera. Un **flujo** (*stream*) es la secuencia de datos que el programa envía a un fichero o recibe de él. `serialVersionUID` identifica la versión de la estructura serializada de la clase y ayuda a comprobar compatibilidades al leer.

6.1 Representar un club mediante una clase

Los campos `id`, `nombre` y `ciudad` se guardarán juntos dentro de un objeto `Club`. Un **constructor** crea el objeto con esos datos; los métodos `get...()` permiten consultarlos y `setCiudad(...)` permite cambiar la ciudad.

JAVA

```
import java.io.Serializable;

public class Club implements Serializable {
    private static final long serialVersionUID = 1L;

    private int id;
    private String nombre;
    private String ciudad;

    public Club(int id, String nombre, String ciudad) {
        this.id = id;
        this.nombre = nombre;
        this.ciudad = ciudad;
    }

    public int getId() { return id; }
    public String getNombre() { return nombre; }
    public String getCiudad() { return ciudad; }
    public void setCiudad(String ciudad) { this.ciudad = ciudad; }

    @Override
    public String toString() {
        return id + " | " + nombre + " | " + ciudad;
    }
}
```

6.2 Guardar y recuperar una lista de objetos

Guardaremos **la lista completa como un único objeto**. Esto evita el problema de añadir sucesivas cabeceras binarias al intentar abrir repetidamente `ObjectOutputStream` en modo añadido.

`Files.newOutputStream(ruta)` abre el destino para escribir bytes; `Files.newInputStream(ruta)` lo abre para leer. `writeObject(lista)` escribe la lista serializada y `readObject()` recupera un objeto cuyo tipo debe comprobarse antes de utilizarlo. Además de `IOException`, la lectura puede lanzar `ClassNotFoundException` si Java no encuentra la clase necesaria para reconstruir el objeto.

Ejemplo: guardar tres clubes

JAVA

```
import java.io.IOException;
import java.io.ObjectOutputStream;
import java.nio.file.Files;
import java.nio.file.Path;
import java.util.ArrayList;
import java.util.List;

public class GuardarBinario {
    public static void main(String[] args) {
        Path ruta = Path.of("datos", "clubes.dat");
        List<Club> clubes = new ArrayList<>();
        clubes.add(new Club(1, "Granada CF", "Granada"));
        clubes.add(new Club(2, "Málaga CF", "Málaga"));
        clubes.add(new Club(3, "Cádiz CF", "Cádiz"));

        try {
            Files.createDirectories(ruta.getParent());
            try (ObjectOutputStream salida = new ObjectOutputStream(
                Files.newOutputStream(ruta))) {
                salida.writeObject(clubes);
            }
            System.out.println("Clubes guardados en " + ruta);
        } catch (IOException e) {
            System.err.println("Error al guardar: " + e.getMessage());
        }
    }
}
```

Ejemplo: recuperar los objetos

JAVA

```
import java.io.IOException;
import java.io.ObjectInputStream;
import java.nio.file.Files;
import java.nio.file.Path;
import java.util.List;

public class LeerBinario {
    public static void main(String[] args) {
        Path ruta = Path.of("datos", "clubes.dat");
        if (Files.notExists(ruta)) {
            System.out.println("No hay clubes guardados.");
            return;
        }

        try (ObjectInputStream entrada = new ObjectInputStream(
            Files.newInputStream(ruta))) {
            Object dato = entrada.readObject();
            if (!(dato instanceof List<?> lista)) {
                System.out.println("El archivo no contiene una lista.");
                return;
            }
            for (Object elemento : lista) {
                if (!(elemento instanceof Club club)) {
                    System.out.println("Se ha encontrado un dato inesperado.");
                    return;
                }
                System.out.println(club);
            }
        } catch (IOException | ClassNotFoundException e) {
            System.err.println("No se pudo recuperar: " + e.getMessage());
        }
    }
}
```

`instanceof` comprueba el tipo de un objeto. En `dato instanceof List<?> lista`, `<?>` significa que en ese punto todavía no hemos especificado el tipo de los elementos; por eso el ejemplo **también comprueba cada elemento** antes de tratarlo como `Club`.

Precaución: no deserialices archivos descargados o de origen desconocido. Este mecanismo se usa aquí con archivos producidos por el propio proyecto y bajo tu control. Cambiar la estructura de `Club` puede impedir leer ficheros binarios antiguos.

Actividades

1. Añade al menú «Guardar en binario» y «Cargar desde binario». Comprueba que los clubes persisten después de cerrar Java.
2. Cambia la ciudad de un objeto, vuelve a guardar la lista completa y comprueba que el cambio aparece al leerla de nuevo.
3. Exporta los objetos recuperados a TXT y contrasta que `id`, `nombre` y `ciudad` coinciden.

7. Acceso aleatorio a registros de tamaño fijo

Con un fichero secuencial, para llegar al último club normalmente debemos recorrer los anteriores. El **acceso aleatorio** permite movernos directamente a una posición del archivo, pero primero hay que conocer cómo se distribuyen los bytes.

`RandomAccessFile` abre un fichero de forma que podemos leer y escribir en posiciones concretas. El modo `"rw"` permite lectura y escritura; `"r"` solo lectura. `seek(posicionEnBytes)` mueve el cursor interno a la posición indicada. `length()` devuelve la longitud total del fichero en bytes. `readInt()` y `writeInt()` leen y escriben un `int` de **4 bytes**; `readBoolean()` y `writeBoolean()` utilizan **1 byte**; `readChar()` y `writeChar()` utilizan **2 bytes** por unidad `char` de Java.

7.1 Diseñar un registro con tamaño conocido

Cada registro de nuestro fichero tendrá un tamaño fijo:

Campo	Tamaño
ID (<code>int</code>)	4 bytes
Activo (<code>boolean</code>)	1 byte
Nombre (24 unidades <code>char</code>)	48 bytes
Total	53 bytes

Para saltar al **registro número 2** contando desde cero, calculamos $2 \times 53 = 106$: el registro empieza en el byte 106. **La posición del registro no es su ID**. El ID es un dato almacenado dentro del registro, mientras que la posición es su lugar físico dentro del archivo.

Los nombres cortos se rellenan con el carácter especial `\0` hasta completar las 24 unidades `char`; no se guardará la ciudad en este ejemplo. Un nombre de más de 24 unidades se rechaza, en lugar de cortarlo silenciosamente. Los caracteres Unicode fuera del plano básico pueden ocupar dos unidades `char`, por lo que el límite no equivale siempre a 24 símbolos visibles.

Para entender el código que viene, necesitas cinco elementos más: `RUTA.toFile()` convierte un `Path` en un `File`, que es el tipo que espera el constructor de `RandomAccessFile`; `nombre.charAt(i)` obtiene el `char` de la posición `i`; `StringBuilder` permite construir una cadena poco a poco y `append(...)` añade un carácter; `IllegalArgumentException` comunica que se ha recibido un argumento incorrecto; y el operador `%` calcula el resto de una división, que utilizaremos para detectar un archivo que no tenga un número completo de registros.

Ejemplo: escribir y leer registros completos

JAVA

```
import java.io.IOException;
import java.io.RandomAccessFile;
import java.nio.file.Files;
import java.nio.file.Path;
```



```

public class ArchivoAleatorio {
    private static final int LONGITUD_NOMBRE = 24;
    private static final int TAM_REGISTRO = 4 + 1 + 24 * 2;
    private static final Path RUTA = Path.of("datos", "clubes.idx");

    public static void anadir(int id, String nombre) throws IOException {
        if (nombre.length() > LONGITUD_NOMBRE) {
            throw new IllegalArgumentException("Nombre demasiado largo");
        }
        Files.createDirectories(RUTA.getParent());
        try (RandomAccessFile archivo =
            new RandomAccessFile(RUTA.toFile(), "rw")) {
            archivo.seek(archivo.length());
            archivo.writeInt(id);
            archivo.writeBoolean(true);
            for (int i = 0; i < LONGITUD_NOMBRE; i++) {
                archivo.writeChar(i < nombre.length()
                    ? nombre.charAt(i) : '\0');
            }
        }
    }

    public static void mostrar() throws IOException {
        try (RandomAccessFile archivo =
            new RandomAccessFile(RUTA.toFile(), "r")) {
            if (archivo.length() % TAM_REGISTRO != 0) {
                throw new IOException("El archivo contiene un registro incompleto");
            }
            for (long pos = 0; pos < archivo.length();
                pos += TAM_REGISTRO) {
                archivo.seek(pos);
                int id = archivo.readInt();
                boolean activo = archivo.readBoolean();
                StringBuilder nombre = new StringBuilder();
                for (int i = 0; i < LONGITUD_NOMBRE; i++) {
                    char letra = archivo.readChar();
                    if (letra != '\0') nombre.append(letra);
                }
                System.out.println(id + " | " + nombre
                    + " | Activo: " + activo);
            }
        }
    }

    public static void main(String[] args) {
        try {
            anadir(10, "Granada CF");
            anadir(20, "Málaga CF");
            anadir(30, "Cádiz CF");
            mostrar();
        } catch (IOException e) {
            System.err.println(e.getMessage());
        }
    }
}

```

Si `length() % 53` no es cero, el archivo no contiene un número completo de registros.

Ejemplo: desactivar un registro sin reescribir todo el fichero

La marca «activo» está **cuatro bytes después** del comienzo del registro, porque el campo ID ocupa los cuatro primeros. Antes de modificar comprobaremos que la posición exista.

JAVA

```

public static void desactivarPosicion(int numeroRegistro)
    throws IOException {
    try (RandomAccessFile archivo =
        new RandomAccessFile(RUTA.toFile(), "rw")) {
        long inicio = (long) numeroRegistro * TAM_REGISTRO;
        if (numeroRegistro < 0 ||
            inicio + TAM_REGISTRO > archivo.length()) {
            throw new IllegalArgumentException("Posición inexistente");
        }
        archivo.seek(inicio + 4);
        archivo.writeBoolean(false);
    }
}

```

Este método se añade **dentro de la clase `ArchivoAleatorio`**. Desactivar no elimina físicamente el registro: solo cambia su marca booleana. Antes de probarlo, ejecuta una sola vez el programa que escribe los registros o borra expresamente el archivo de prueba anterior, porque `anadir(...)` siempre agrega datos al final.

Actividades

1. Guarda tres registros y comprueba que el fichero ocupa $3 \times 53 = 159$ bytes.
2. Desactiva el segundo registro (posición 1) y comprueba que su marca cambia, pero el tamaño total no.
3. Implementa `cambiarNombrePosicion(...)`, manteniendo exactamente 24 unidades `char` en el campo nombre.
4. Compara la modificación del nombre en este fichero con la modificación de una línea en TXT: ¿en cuál necesitas reescribir todos los registros?

8. Intercambio de datos con XML

Guardar datos no sirve solo para recuperarlos desde nuestro propio programa: a menudo necesitamos intercambiarlos con otras aplicaciones. **XML** (*eXtensible Markup Language*) es un formato de texto estructurado mediante **etiquetas** y **atributos**.

XML

```

<?xml version="1.0" encoding="UTF-8"?>
<clubes>
  <club id="1">
    <nombre>Granada CF</nombre>
    <ciudad>Granada</ciudad>
  </club>
</clubes>

```

Aquí `<clubes>` es la **etiqueta raíz** que contiene todos los registros; cada `<club>` es un **elemento**; `id="1"` es un **atributo** del elemento; y `<nombre>` y `<ciudad>` son elementos hijos. El texto especial de XML, como `&` o `<` dentro de un valor, debe representarse correctamente. Por eso crearemos el documento con una API en vez de construir etiquetas concatenando cadenas.

8.1 Crear XML mediante DOM

DOM (*Document Object Model*) representa un XML en memoria como un árbol de nodos. La API del JDK incluye `DocumentBuilderFactory` para crear un documento, `Document` para representarlo y `Element` para construir etiquetas. `createElement("nombre")` crea un elemento; `setTextContent(...)` pone su texto; `appendChild(...)` lo añade como hijo de otro elemento; `setAttribute(...)` añade un atributo.

Para convertir el árbol en un fichero utilizaremos un `Transformer` obtenido de `TransformerFactory`. `DOMSource` indica cuál es el árbol de origen y `StreamResult` dónde debe escribirse. `OutputKeys.ENCODING` permite indicar UTF-8 en la salida.

En este ejemplo, `String[][]` es una matriz de cadenas usada para introducir datos iniciales. El método `main` declara `throws Exception` para que se vean los errores mientras se ejecuta este ejemplo mínimo; en el gestor final los capturaremos y mostraremos un mensaje claro.

Ejemplo: exportar dos clubes a XML

JAVA

```
import java.nio.file.Files;
import java.nio.file.Path;
import javax.xml.parsers.DocumentBuilderFactory;
import javax.xml.transform.OutputKeys;
import javax.xml.transform.Transformer;
import javax.xml.transform.TransformerFactory;
import javax.xml.transform.dom.DOMSource;
import javax.xml.transform.stream.StreamResult;
import org.w3c.dom.Document;
import org.w3c.dom.Element;

public class ExportarXml {
    public static void main(String[] args) throws Exception {
        Document documento = DocumentBuilderFactory.newInstance()
            .newDocumentBuilder().newDocument();
        Element raiz = documento.createElement("clubes");
        documento.appendChild(raiz);

        String[][] datos = {
            {"1", "Granada CF", "Granada"},
            {"2", "Málaga CF", "Málaga"}
        };
        for (String[] fila : datos) {
            Element club = documento.createElement("club");
            club.setAttribute("id", fila[0]);
            Element nombre = documento.createElement("nombre");
            nombre.setTextContent(fila[1]);
            Element ciudad = documento.createElement("ciudad");
            ciudad.setTextContent(fila[2]);
            club.appendChild(nombre);
            club.appendChild(ciudad);
            raiz.appendChild(club);
        }

        Path ruta = Path.of("datos", "clubes.xml");
        Files.createDirectories(ruta.getParent());
        Transformer transformador = TransformerFactory.newInstance()
            .newTransformer();
        transformador.setOutputProperty(OutputKeys.ENCODING, "UTF-8");
        transformador.setOutputProperty(OutputKeys.INDENT, "yes");
        transformador.transform(
            new DOMSource(documento), new StreamResult(ruta.toFile()));
        System.out.println("XML guardado en " + ruta.toAbsolutePath());
    }
}
```

El XML generado contiene dos elementos `club` dentro de la raíz `clubes`. Comprueba que puedes abrirlo con un editor y que su contenido corresponde a los datos iniciales.

8.2 Leer XML sin perder datos si está mal formado

Para leer un XML, el mismo `DocumentBuilderFactory` crea un parser mediante `newDocumentBuilder()` y después `parse(...)` reconstruye el árbol. `getElementsByTagName("club")` devuelve una colección `NodeList` de elementos `club`; `item(i)` obtiene uno de ellos; `getAttribute("id")` recupera su atributo. Un documento externo puede contener declaraciones que intenten acceder a archivos o recursos ajenos; por eso desactivaremos el uso de DTD y entidades externas en este ejercicio.

Regla de importación: primero lee y valida todos los registros en una lista temporal; **solo después** sustituye o fusiona la lista de clubes de la aplicación. Así, un XML incompleto no destruye los datos que ya estaban guardados.

Al leer, `getLength()` indica cuántos elementos hay en un `NodeList`; `getTextContent()` devuelve el texto de un elemento; `getAttribute("id")` devuelve el texto del atributo `id`. `trim()` elimina espacios al principio y al final de una cadena. Antes de intentar leer un hijo con `item(0)` comprobaremos que existe, ya que de lo contrario recibiríamos `null`.

Ejemplo: importar clubes desde XML

El siguiente código utiliza la clase `Club` creada en el apartado 6. Desactiva las DTD y entidades externas antes de leer el documento, valida sus campos y muestra los clubes **solo cuando el archivo completo se ha leído correctamente**:

JAVA

```
import java.nio.file.Path;
import java.util.ArrayList;
import java.util.List;
import javax.xml.parsers.DocumentBuilderFactory;
import org.w3c.dom.Document;
import org.w3c.dom.Element;
import org.w3c.dom.NodeList;

public class ImportarXml {
    public static void main(String[] args) {
        Path ruta = Path.of("datos", "clubes.xml");
        try {
            DocumentBuilderFactory fabrica =
                DocumentBuilderFactory.newInstance();
            fabrica.setFeature(
                "http://apache.org/xml/features/disallow-doctype-decl",
                true);
            fabrica.setFeature(
                "http://xml.org/sax/features/external-general-entities",
                false);
            fabrica.setFeature(
                "http://xml.org/sax/features/external-parameter-entities",
                false);
            fabrica.setIncludeAware(false);
            fabrica.setExpandEntityReferences(false);

            Document documento = fabrica.newDocumentBuilder()
                .parse(ruta.toFile());
            NodeList elementos = documento.getElementsByTagName("club");
            List<Club> importados = new ArrayList<>();

            for (int i = 0; i < elementos.getLength(); i++) {
                Element clubXml = (Element) elementos.item(i);
```

```

        NodeList nombres = clubXml.getElementsByTagName("nombre");
        NodeList ciudades = clubXml.getElementsByTagName("ciudad");
        if (nombres.getLength() != 1 ||
            ciudades.getLength() != 1) {
            throw new IllegalArgumentException(
                "Faltan campos en el club " + (i + 1));
        }
        int id = Integer.parseInt(clubXml.getAttribute("id"));
        String nombre = nombres.item(0).getTextContent().trim();
        String ciudad = ciudades.item(0).getTextContent().trim();
        if (id <= 0 || nombre.isEmpty() || ciudad.isEmpty()) {
            throw new IllegalArgumentException(
                "Datos inválidos en el club " + (i + 1));
        }
        for (Club anterior : importados) {
            if (anterior.getId() == id) {
                throw new IllegalArgumentException("ID duplicado: " + id);
            }
        }
        importados.add(new Club(id, nombre, ciudad));
    }

    for (Club club : importados) {
        System.out.println(club);
    }
} catch (Exception e) {
    System.err.println("No se pudo importar: " + e.getMessage());
}
}
}

```

`isEmpty()` comprueba si una cadena está vacía. La conversión `(Element)` indica a Java que el nodo obtenido representa un elemento XML, y permite utilizar sus métodos específicos. En la aplicación final, guarda o incorpora `importados` a la lista principal **después** de terminar las validaciones, nunca dentro del bucle de lectura.

Actividades

1. Exporta la lista de clubes del gestor a XML, incluyendo un `club` por objeto y sus tres campos.
2. Crea una opción «Importar XML»: comprueba que todos los clubes tienen ID válido, nombre y ciudad antes de modificar los datos existentes.
3. Elimina a propósito la etiqueta `ciudad` de un XML de prueba y comprueba que la importación rechaza ese registro sin borrar el fichero original.
4. Realiza la conversión TXT → XML → TXT y verifica que se conservan los IDs, los espacios y las tildes.

9. Intercambio de datos con JSON

JSON (*JavaScript Object Notation*) es otro formato de texto para intercambiar información. Un **objeto JSON** agrupa pares `"clave": valor`, y un **array JSON** agrupa varios valores entre corchetes. Los textos se escriben entre comillas dobles y los números no necesitan comillas.

JSON

```
[
  {"id": 1, "nombre": "Granada CF", "ciudad": "Granada"},
  {"id": 2, "nombre": "Málaga CF", "ciudad": "Málaga"}
]
```

Construir JSON concatenando cadenas puede producir errores con comillas, saltos de línea o caracteres especiales. Utilizaremos una librería que convierta directamente objetos Java a JSON y viceversa.

9.1 Qué es una dependencia Maven

Maven es una herramienta para gestionar proyectos Java. El archivo `pom.xml` indica qué librerías necesita el proyecto. Cada **dependencia** identifica una librería que Maven incorpora para que podamos utilizar sus clases. Para JSON emplearemos **Jackson**: su clase `ObjectMapper` traduce objetos Java a JSON y también lee JSON para reconstruir objetos.

Crea un proyecto Maven con Java 17 y añade al bloque `<dependencies>` de `pom.xml` esta dependencia del módulo de Jackson que realiza el mapeo:

XML

```
<dependency>
  <groupId>com.fasterxml.jackson.core</groupId>
  <artifactId>jackson-databind</artifactId>
  <version>2.18.3</version>
</dependency>
```

Si el `pom.xml` todavía no tiene `<dependencies>`, crea ese bloque dentro de `<project>` y coloca la dependencia anterior dentro de él. Después solicita a tu IDE que **recargue Maven**. La primera vez necesitarás conexión para descargar la librería si no se encuentra en el equipo.

9.2 Pasar de objetos Java a JSON y recuperarlos

Un `record` es una forma abreviada de declarar una clase cuyo objetivo principal es almacenar datos; en este ejemplo, `ClubJson` tendrá `id`, `nombre` y `ciudad`. `List.of(...)` crea una lista con los elementos indicados. `writerWithDefaultPrettyPrinter()` hace que el JSON se escriba con saltos e indentación para poder leerlo cómodamente.

`ObjectMapper.readValue(...)` reconstruye objetos desde un fichero JSON. `TypeReference<List<ClubJson>>` le informa de que esperamos una **lista de clubes** y no un único objeto. El `record` genera automáticamente los métodos `id()`, `nombre()` y `ciudad()` para consultar sus campos.

JAVA

```
import com.fasterxml.jackson.core.type.TypeReference;
import com.fasterxml.jackson.databind.ObjectMapper;
import java.nio.file.Files;
import java.nio.file.Path;
import java.util.List;

public class ClubesJson {
    public record ClubJson(int id, String nombre, String ciudad) {}

    public static void main(String[] args) throws Exception {
        Path ruta = Path.of("datos", "clubes.json");
        Files.createDirectories(ruta.getParent());
        ObjectMapper mapper = new ObjectMapper();

        List<ClubJson> clubes = List.of(
            new ClubJson(1, "Granada CF", "Granada"),
            new ClubJson(2, "Málaga CF", "Málaga"));

        mapper.writerWithDefaultPrettyPrinter()
            .writeValue(ruta.toFile(), clubes);

        List<ClubJson> recuperados = mapper.readValue(
            ruta.toFile(), new TypeReference<List<ClubJson>>() {});
        for (ClubJson club : recuperados) {
            System.out.println(club.id() + " | " + club.nombre()
                + " | " + club.ciudad());
        }
    }
}
```

Al igual que en XML, **no sustituyas los datos guardados hasta comprobar que la lectura y las validaciones han terminado sin errores.**

Actividades

1. Exporta los clubes de tu aplicación al archivo `clubes.json` e impórtalos después.
2. Modifica la ciudad de un club, exporta la lista de nuevo y comprueba el cambio abriendo el JSON con un editor de texto.
3. Prueba un fichero JSON con una coma o una llave incorrecta. El programa debe mostrar un error y conservar los clubes que ya tenía.
4. Compara el TXT inicial, el XML y el JSON: identifica qué información contiene cada uno y verifica que se puede recuperar el mismo conjunto de clubes.

10. Organización del acceso a datos

En el gestor ya hemos escrito código para pedir opciones al usuario, validar clubes y acceder a archivos. Conviene separar esas responsabilidades: el **programa principal** gestiona el menú y decide qué quiere hacer; una clase de **acceso a datos** sabe cómo guardar, buscar o borrar la información.

Una **interfaz** Java puede definir un contrato: nombres de métodos y resultados que deben ofrecer distintas implementaciones. Por ejemplo, podríamos guardar clubes hoy en TXT y en la siguiente unidad en una base de datos sin tener que modificar todas las opciones del menú.

JAVA

```
import java.io.IOException;
import java.util.List;

public interface RepositorioClubes {
    void guardarTodos(List<Club> clubes) throws IOException;
    List<Club> cargarTodos() throws IOException;
}
```

`void` significa que `guardarTodos` no devuelve un valor; `List<Club>` indica que `cargarTodos` devuelve una lista de clubes. La palabra `throws IOException` advierte a quien llama al método de que una operación de disco puede fallar y debe gestionarse.

Una clase `RepositorioTexto` puede implementar ese contrato leyendo y escribiendo TXT; otra clase `RepositorioBinario` podría hacerlo utilizando objetos serializados. **No necesitamos construir dos programas completos:** basta con cambiar qué repositorio recibe el menú.

Actividades

1. Extrae los métodos de lectura y escritura del menú a una clase separada, de modo que `Main` no utilice directamente `Files`.
2. Haz que una clase implemente `RepositorioClubes` con el fichero TXT.
3. Sustituye el repositorio TXT por otro binario conservando las mismas operaciones visibles para el usuario.

11. Proyecto de unidad: gestor de clubes

Desarrolla una aplicación Java que recuerde los datos de los clubes cuando se cierra y se vuelve a abrir. Debe permitir introducir clubes, mostrarlos, buscar por ID, cambiar su ciudad, borrarlos, crear y restaurar copias, e importar o exportar información.

11.1 Opciones del programa

DATOS

```
GESTOR DE CLUBES
1. Añadir club
2. Listar clubes
3. Buscar por ID
4. Modificar ciudad
5. Borrar club
6. Guardar en TXT
7. Guardar o cargar en binario
8. Exportar o importar XML
9. Exportar o importar JSON
10. Crear o restaurar copia de seguridad
0. Salir
```

11.2 Condiciones que debe cumplir

- Al cerrar y volver a ejecutar el programa, los datos guardados siguen disponibles.

- El programa no inserta dos clubes con el mismo ID y no falla al buscar o borrar un ID inexistente.
- La lectura detecta datos incompletos o mal formados sin destruir información válida.
- Antes de sobrescribir o restaurar información importante se crea una copia o se solicita confirmación, según corresponda.
- El código que muestra el menú está separado del código que accede a los ficheros.

11.3 Comprobaciones finales

Prueba el programa con tres clubes, modifica uno, guarda los cambios y cierra Java. Después vuelve a abrirlo y comprueba que se recuperan exactamente esos tres clubes con la ciudad actualizada. Repite la importación de un mismo archivo y verifica que no aparecen IDs duplicados. Por último, introduce un fichero incorrecto y comprueba que un error de lectura no borra los datos válidos anteriores.

Al terminar la unidad sabrás conservar información en ficheros, recuperar objetos, localizar registros y convertir datos entre varios formatos. En la siguiente unidad la misma idea de persistencia se aplicará a una base de datos relacional.